Linux on Android + HA

Install Full Linux on ANY Android (No Root) + PC Gaming | Hardware Acceleration

Outlier score : 100x

Script:

Today, I have a 6-year-old Android phone, and right now it's running a full desktop Linux distribution. I can multitask, write code in Visual Studio Code, run a Python server, and even play AAA PC games. If I connect it to a monitor or a PC, I can use it on a bigger screen just like a normal desktop. And the best part, everything works without root. So, how is this possible? Let me walk you through it. Right now you're looking at a full Linux desktop environment. This is XFCE running on top of Android using Termox and Termox X11. And this part is important. This is not emulation. Android already runs on a Linux kernel. So we're

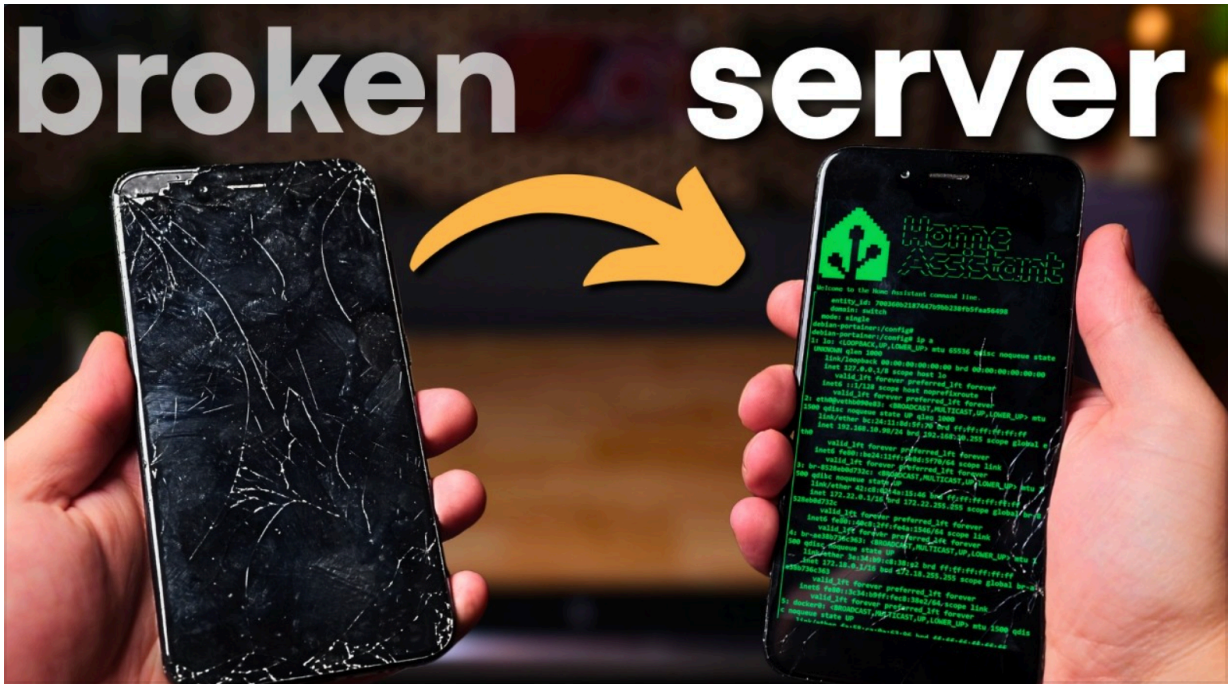
simply using that same kernel to run a desktop Linux user space environment. That's why performance is much better than you would expect. Here's Firefox opening websites in full desktop mode. Audio works as well which normally does not work in a default term setup. Since this is XFCE, we get the Thunar file manager. I can browse Linux directories normally and I can also access Android storage from inside this environment. So it feels properly integrated. Now let's test something practical. This is Visual Studio Code running directly on the phone. I installed it using the Termox user repository, also known as TUR. I've installed Python as well. And here I'm running a small Flask server. And yes, I can access that server directly from the Android browser. That means this isn't just a demo environment. You can actually develop inside it. It comes with all the essential Linux tools you'd expect. You can install applications directly from the terminal using standard package commands. Since this runs inside termox, we don't have snap or flatpack. But tur gives us access to many desktop applications. For example, I can search for a package using pkg search followed by the application name and install it using pkg install followed by the name. That's how I installed Blender, Visual Studio Code, and Python. The terminal behaves exactly like a regular Linux system. You can manage files, install dependencies, run servers, compile programs. Everything works as expected. Typing is also practical. In landscape mode, the onscreen keyboard may cover a large portion of the screen, but rotating the phone adjusts the layout automatically, making it much easier to work. If you connect a physical keyboard, it feels even closer to a real desktop experience. Now, let's push it further. I first installed Wine to run Windows applications. It worked fine for lighter software. But when I tried installing heavier AAA games, it started crashing. So I switched to Winlater. Winlater is an Android application that allows you to run Windows x64 applications using Wine and Translation layers. It creates a Linux- based container and runs Windows software inside it. And this is Deadpool running on my 6-year-old phone. Is it perfect? No. Is it playable? Yes. So this phone is now acting as a Linux workstation, a development machine, and even a Windows gaming device, all without root. But performance depends heavily on your processor. Now let's talk about installation. First, download two applications, Termox and Termox X11. Download both from FDroid. Do not use the Play Store version. After installing them, there are two important things you must do before starting. First, open Termox and run the command termox- setup-

storage. This will ask for storage permission. Allow it so the Linux environment can access your internal storage. Second, go to your phone's developer options and disable child process restrictions. Some devices limit background processes and that can break the Linux session. Disabling this ensures everything runs properly. Now, go back to Termox. First things first, we need to make sure everything is up to date. Go ahead and type a update and upgrade. Hit enter and just let it do its thing. After that, install git by typing apt install git. Once Git is installed, next head down to the description and click the link to my GitHub repo. Once you're there, copy the URL. Then go back to your terminal, type git clone, add a space, and paste the link. Press enter to clone the repository in Termix. After it finishes downloading, type ls to see the folder. Then move into the cloned directory using the change directory command. Inside that folder, you'll see the installation script. Before running it, make it executable by typing chode +x followed by the script name. Run it by typing dot slash followed by the script name. Now the script gives us the option to choose our desktop environment. If you have a Snapdragon device, you're in a better position. Snapdragon processors use adreno GPU and there's an open- source driver called Turnip specifically built for Arduino. My installation script automatically detects Snapdragon devices and enables turnip for hardware acceleration. If you're using MediaTek or Exynos, the setup will still work, but it will fall back to software rendering. In that case, it's better to use lighter desktop environments like XFCE or LXQT and avoid heavy 3D gaming. If your device supports hardware acceleration and has more than 8 GB of RAM, you can experiment with heavier desktop environments. If your device relies on software rendering, choose XFCE or LXQT for better stability. In my case, even though my phone supports hardware acceleration, I chose XFCE because it's an older device and XFCE was the most stable during testing. Installation takes around 7 to 8 GB of storage. So, make sure you have enough space and keep your screen awake during the process. When the install finishes, it drops two scripts into your home folder. One to start it and one to stop it. Type cd to go home. From here, just type dot slash then the start script and hit enter to boot up Linux. Before starting, make sure Termix X11 is open in the background. Now, open Termox X11. Within seconds, your full Linux desktop will appear. And there we go. We've successfully installed Linux on top of Android. And best of all, we've got full hardware acceleration. If you want Linux to start automatically every time you open Termox, you

can add the start command inside your Bash RC file. For gaming, start by installing Windlater from its official GitHub page. Once it's installed, open the app and create a new container. When it asks for the architecture, choose X64. Next, under graphics, select turnip if your device has an adreno GPU. If not, choose Ver instead. For the direct Xrapper, select DXVK. And finally, set the resolution to 1280x 720. This usually gives you better and more stable performance on most phones. When the container opens, the D drive inside Winlater maps to your Android downloads folder. Place your game files there and install them normally. If your game is compressed, extract it using Android first instead of inside the container. It's much faster that way. So, that's it. This is how I turned a normal Android phone into a functional desktop machine capable of running Linux, handling real development work, and even playing Windows PC games. And the most impressive part is that this works without root, without modifying the system, and without flashing anything. It's simply using Android's Linux foundation the way it was meant to be used. Now, is it going to replace a high-end PC? No. But for development, experimentation, learning Linux, or even portable computing, this setup is surprisingly powerful. If you try this on your device, let me know your processor and how it performs. I'm especially curious how different chipsets handle hardware acceleration.

I Turned This Broken Phone Into A Home Server

Outlier score : 4x



Script:

This old shattered Android phone might seem pretty useless, but I managed to breathe some new life into it by turning it into a compact home server that draws almost no power and can run something called Home Assistant to automate all of my smart home devices. And the whole process was incredibly easy. I'm just kidding. It was a huge pain in the butt, but I did it. And yes, I do run a YouTube channel about tech and computers, but I'm actually not as competent with stuff like this as you might think. So, yeah, this is by no means a plug-and-play solution, but it was definitely doable. And in the end, I think it was a great way to repurpose some older tech in a surprisingly useful way. So, let me show you how I did it. [Music] If you're wondering why I would attempt to turn an old Android phone into a little server, well, you're probably new to the channel. So, hey, welcome. This video concept has been one of the most requested topics I've seen basically since I created the channel. And it makes sense. I'm sure many of you have upgraded phones at some point and for one reason or another held on to your previous phone which possibly landed in a junk drawer or something. And well, it would probably be pretty cool if you could put it to use somehow. Now, if you have watched my channel, you know I love self-hosting or running your own services on your own servers. And while I would normally use a PC of some kind for that task, why not a phone? Well, as it turns out,

probably because it's not nearly as straightforward. But if you could, it might work great. Phones are designed to draw very little power. something that's nice to have if you're running a server 24/7. And while most older phones probably don't have a ton of processing power, it's probably going to be plenty for running simple things like Home Assistant or ABG Guard or maybe even a VPN server. Now, if you are needing a VPN and you can't get one running on a janky phone server setup thing, you can always check out Private Internet Access, the sponsor of today's video. Even if you do have your own VPN, PIA is a great option to have. They have tons of servers across 91 different countries, making it easy to access the information and content you need, regardless of where you are. I love that I can run PA on as many devices as I want. From my phone to my desktop PC. I can even run it on my laptop that I'm using right now to write this script, which is running Linux Mint. But one of the main ways I use PA is with a Docker container called Gluten. It integrates really well and makes it super easy to route traffic from a bunch of other containers through PIA servers. I've been using PIA personally for over a year now and I've experienced zero issues, slowdowns, or anything like that. It's been very rock solid. And I also appreciate that they have a strict no logs policy that's been independently verified by thirdparty audits. So if you're looking for an affordable, featurerich, and rocksolid VPN, check out PIA by using my link down in the description. With that, you can get 83% off plus 4 months for free. There are a few different ways that you could go about converting an older Android phone into a server of sorts. The first, and probably the most obvious, is to just run Android. For example, you can run a terminal emulator app called Termix, which well, it gives you a terminal, but it also creates a virtual Linux environment. This can probably do a lot of things, but if you're looking to run something like Docker, for example, you're probably going to have to root your device and do some dirty work. I considered this route, but well, basically, I have no experience with Android or Android devices. I'm also just a creature of habit, and when it comes to server projects, I typically just like installing a Linux dro that I'm comfortable with. If I want to repurpose an old office PC, Debian. A 2012 MacBook Pro? Debian. An unsupported NAS from Netgear. Yep, also Debian. But sadly, you can't just go installing Debian onto an Android phone. I tried. Come on. There are a handful of projects out there aimed at getting Linux running on old Android devices. But the most obvious choice seem to be something

called Postmarket OS. Postmarket OS is actually pretty awesome with the primary goal of extending the life of devices and giving users control of their phones and data. It's a really exciting prospect, but even they are honest about the current state of the software. It's definitely for enthusiasts and tinkerers, not for mainstream audiences, at least at the moment. But I wasn't looking for a daily driver phone. I was pretty much just looking to get something like Docker working. So, I figured this might be a great choice. It's not Debian, but it's an actual Linux distro with a kernel and everything. Plus, the documentation seemed to be really solid. Really, the biggest downside of Postmarket OS is that, well, at the moment, there are only a handful of devices that have at least what they call community level support, which means the device mostly functions as intended. So, there's a really good chance that your old phone might fall under the testing category or just might not be supported at all. Now, as I said earlier, I don't have much experience with Android devices. I pretty much just always use an iPhone. So, obviously, I don't already have an old Android phone. To rectify that situation, I hopped on eBay and bought this beat up Xiaomi Mi A1 for like $30. Well, it was what I thought was a Mi A1. It turns out this is actually a Mi5X, which is basically the exact same phone, but with this one, to unlock the bootloader, you have to create a Xiaomi account and then wait like 15 days, which I considered doing, but when I actually tried to do it, the link to the website was just like broken. And I just gave up and spent another $30 on this beat up Mi A1. This thing isn't awesome. It's like 8 years old, has 4 GB of RAM, but it did seem to be well supported with Postmarket OS, and I got it for 30 bucks, so I gave it a shot. Now, I do want to be clear, this is not a tutorial. I did stumble my way through this and eventually ended up with a working finished product, but it was a very rough process. In fact, I actually recorded the whole process and was going to show it live of how I installed Postmarket OS, but it was so clunky and I was going back and forth between different things trying to figure out what the heck I was doing. And so rather than showing that whole process uh and it being really confusing, I'm just going to talk through what I did because I think that'll make more sense. But once again, this is not a tutorial. I'm just showing you the process that I went through and maybe it'll help you if you want to do something similar or just uh be entertaining to watch. So following the instructions for the Postmarket install, I started just making sure I have everything prepared. First, I installed Fast Boot, which is part of the Android SDK platform

tools package. And you need this to be able to actually flash images to your phone. You can also install PM Bootstrap or Postmarket Bootstrap, but this is only needed if you want to build your own images. You can just install pre-made images as long as they exist for your device. Now, I was using my laptop, which is running Linux Mint for this. And I do advise that if you can try to use some sort of Linux DRO. I believe that you can use Windows or Mac OS to run fast boot and install the pre-built images. But if you want to run PM Bootstrap to build your own, you're going to need to run Linux. Also, all of the documentation sort of assumes you're running Linux, so that's probably going to make following the instructions a bit easier as well. Now, I did mention pre-made images, so for me, I was able to find the ones that existed for the Mi A1. And here, there were a different variety of user interfaces, and I went with KDE Plasma mobile because I don't know why, it just sounded good. There are two files here. One of them is the main image, and then there's also a bootloader image for the LKC second bootloader, which is required for my device at least. To get my phone ready for flashing, I first enabled dev mode and then enabled USB debugging and then booted into fast boot and connected it to my computer. Now, do be aware that how you enable dev mode and get to fast boot is going to be device dependent. So, you'll have to look up that for your specific device. Once I had it connected, I could run fast boot devices to see that it was actually showing up as a device. Now, I did say that Postmarket OS's documentation was pretty good, but the instructions can be a bit clunky if you don't really know what you're doing because there's sort of a main instruction list, but then there's also instructions for your specific device, and you kind of have to hop back and forth between them, and sometimes it's not super clear which step needs to come next. For example, it took me quite a bit to realize I didn't actually need to run PM Bootstrap since I wasn't going to be building my own images. I'm realizing now I probably don't need to run this uh PM Bootstrap at all. I think this was to actually create a custom image. So, I don't think I need to do any of that, which that's probably me just not knowing what I'm doing. But eventually, I did realize that I could just use fast boot to flash the images that I already downloaded. First, I tried flashing the bootloader, but that failed because, well, I realized I never actually unlocked the bootloader. So, I ran fast boot OEM unlock, which gave me this scary warning when the phone rebooted. It also had a typo, which I thought was kind of funny. But after that, I could go back to flash boot

and I was able to flash the bootloader. And then after that, I flashed the main image to the user data partition. Then after a reboot, it booted into Postmarket OS. Hey, thumbnail. >> Now, I will say that at least with the Plasma mobile UI, things were a bit clunky. For example, about a third of the time you'd open the settings app, it would just crash. Uh the web browser just got locked up as soon as you tried to use it. And the shutdown button in the UI just well, it just didn't work. But like I said, I wasn't planning on using this as like an actual phone. I just wanted to use it as a server. And I was able to edit the SSH config to allow for password authentication. That way, I could easily access it over SSH using the Wi-Fi connection. And yeah, Wi-Fi isn't the best when it comes to servers because you typically want them to be reliable. And well, Wi-Fi is not necessarily the most reliable thing. So, I had originally planned to solve that issue with a little adapter like this one, which not only has gigabit networking, but also has power pass through, so we could actually power the phone. Now, the networking on this does work, but because the USB port is only USB 2, I believe, I was only able to get around 230 megabits per second or so. Still, that's better than Wi-Fi when it comes to latency and reliability. But the bigger issue with this was that, well, the power just didn't work. And the reason for this, I believe, is that Xiaomi uses a weird proprietary fast charging protocol. So, rather than using like USB power delivery or one of the other standards that kind of exists out there, they use their own. So, you can just charge this with a standard USB like 5V adapter, but if you want it to charge faster, you have to use a a specific adapter that they provide, I think. So, the issue is that I believe this little adapter is designed to use USB power delivery. And so, it's not compatible with this. And even if I just tried to plug in a standard USB 5V power adapter into this, it didn't work. I think this actually has like a USB power delivery chip on here that's designed to power this. It's not just like a pass through. Now, I did have a plan, or more so a concept of a plan of how I might solve this, but there was a decent chance in doing so I might break the phone. So, I just decided to proceed with Wi-Fi and come back to that issue later. I did get Docker installed, which was actually pretty easy, and started by just spinning up a simple Engine X container. But, I ran into an issue where I couldn't access EngineX from my desktop or other devices on the local network. I could only access it from the phone itself, which as expected, it ended up being an issue with firewall rules. With most of the projects that I've done, I've never really had to mess with NF tables. But uh after

this project, I was much more familiar because well, this whole process of working on this phone was basically just a constant battle with trying to get things working with NF tables. But I was able to get things working after tinkering for a while. I had ended up starting up a home assistant container just using the host networking driver, but I still couldn't access that until I made an NFT rule to accept incoming traffic on port 8123. And then after that, I was able to access Home Assistant and it actually worked great. I was able to discover all the devices on my network. I was able to set things up and it worked pretty much exactly how you would expect. I also tried running some other containers like for example Crafty Controller to run a Minecraft server. But here I ran into a lot of the same issues as before, especially when I tried to do port mapping in Docker. I was still able to use host networking and then make any firewall rules I needed. But when I actually tried to spin up a Minecraft server, I started getting errors with Java. And this actually brings up a big issue you're probably going to run into at some point if you're trying to run different services on old Android devices, which is that uh well, these aren't really that well supported with a lot of software, and you might just run into some issues. I didn't really care all that much to try to dive into figuring out how to get Java runtimes working and such. And at this point, I decided to just narrow my scope and try to really focus on just turning this into a great little Home Assistant server, which yeah, I already did have Home Assistant working and running pretty well, but I still had a few big issues. The first issue was that I was still using a graphical user interface. And this wasn't that big of a deal because the display would turn off after a few minutes to save power. But having a user interface meant I was taking up storage and memory. And with only 64 gigs of storage and 4 gigs of RAM, every little bit counts. The much bigger issue, though, was that I was still just having to use the USB port to charge the battery. Not only did this mean that I couldn't plug in an adapter to have wired networking, but it also meant that I was still running a battery 24/7, which uh would probably lead to a spicy pillow situation. I decided to tackle getting rid of the UI first, and initially started following a guide that I found from Brian's place, which is actually pretty awesome. Not only did he get Docker up and running, but he was actually running the exact same phone that I was. He also didn't seem to run into any of the firewall issues that I was dealing with. And so, I was hoping that if I followed that guide, I might have better luck. But I ended up running into some other issues because

well the guide was a little bit older and there were some things that have changed and so I ended up just trying to build my own image. This was actually pretty easy thanks to PM Bootstrap. You pretty much just run PM Bootstrap init and then follow the instructions here. I pretty much just selected defaults for everything, but I did change the UI to just console only. And I also threw in some extra packages from the Brian's Place guide like Nano and then some networking tools. After building the image, I was able to flash it, actually using Postmarket Bootstrap. And then once it was booted up, I was even able to use USB networking to SSH in. This let me set up my Wi-Fi credentials without trying to get a keyboard working, which was really handy. Everything seemed to be working well, and now I didn't have any overhead from running a user interface. I got Docker installed and was hoping that all of my issues with NFT tables were going to be solved. But well, I actually ran into a new issue. Before, when I was running Plasma, I had issues reaching the Home Assistant web interface. But once I created that one rule, I could access the interface and then Home Assistant had no issues discovering devices on my network. Well, at least as long as I was using the host networking driver. But now, not only did I still have to create that firewall rule to allow connections on port 8123 for the web interface, Home Assistant couldn't find pretty much any of my devices. Now, very long story short, I was able to fix this by adding firewall rules for not only the Home Assistant web interface, but also for MDNS. And then I just accepted all unsolicited UDP packets from at least the local network. It seems like that last rule is pretty much required for every discovery protocol out there. Also, on a side note, I did confirm eventually that NF tables were the source of all of my Docker networking issues. I was a bit worried at a certain point that I was going to have to recompile the kernel or something to get Docker networking working, working, working. Anyway, but after creating rules to just accept all incoming and forwarded traffic, Docker networking worked how you would expect. So, if you are planning on running Docker on Postmarket OS and you run into these issues, it seems like they are all technically solvable. I'm just not the expert to actually figure out how to do that. If you have any tips, feel free to post them down in the comments. Anyway, after getting those rules set up, I had a pretty sweet little home assistant setup. And not only did it run great, it was only drawing a single watt from the wall. But we were still on battery. And I know some people might be thinking, well, why not just take the battery out and still just plug in the USB

cable? But from what I understand, pretty much any modern device isn't going to actually turn on if there's no battery and you just plug in the USB cable. So, after watching a handful of videos and reading some guides, I formulated a plan that I was really hoping wouldn't kill my new little phone server thing. But fortunately, as you might remember, I accidentally bought two of these. So, I had a test subject. Now, I'm not going to show you the process of removing the battery from here because it was it was pretty rough, but I was able to get it out. And then I removed this tiny little controller board, which just has two leads that are connected to the battery. I snipped those off. And then I also stripped just a standard USB power cable. Now, just soldering those two wires straight to that little controller would have been a bad idea because these batteries are expected to output, I think, somewhere between like 3.4 and 4.4 volts. So, the 5 volts coming from a standard USB power adapter would be too much. Now, a smart option would be to buy a little adjustable like bucket converter that could bring that 5 volts down to 4 volts, but I didn't want to wait for one. So, instead, I just grabbed one of these diodes and soldered it in series with the positive terminal. This technically should be enough of a voltage drop to bring that 5 volts down to an acceptable level. Although, it is pretty janky. That being said, after hooking the connector back into the phone, it worked. Not only did it boot up and stay on, but unlike some other phones that I watched in some of the videos and guides where it seemed like it sort of estimated the capacity, this phone didn't seem to really mind that while the battery was discharging, the battery percentage never seemed to drop and it just kept running perpetually, even if I watched some YouTube videos and such. So, once I knew that was working, it was time for the real deal. Now, seeing how awkward it is for me to get this phone open is probably pretty tough to watch for anyone who knows what they're doing. But surprisingly, this was much better than the first round. I eventually got the cover off and also removed this little piece to get to the battery. And while I was hoping it was going to have some nice little pull tabs to remove the adhesive like the other phone did, but it did not have those. So, instead, I just had to use some isopropyl alcohol to eventually wear down the adhesive. And I was able to get the battery out without bending it way too much. Now, I wanted to preserve at least one battery from these phones, so I decided to reuse the battery controller from the previous phone. This time, I did a bit better with my soldering job and ended up with a slightly tidier

result. And I also put down some captain tape on the inside of the phone just to help prevent any shorts. After that, I plugged in the battery connector, hit the power button, and nothing. It just wouldn't turn on for like 2 minutes, and then it just did and started working. And I don't know why, but hey, it was working. And I could actually see in the command line that the capacity was showing up as 99% even though it was discharging and the voltage was within the acceptable range. To clean things up a bit, well, I just taped down the cable with some electrical tape because I couldn't find my hot glue gun. And then I also drilled a hole in the back of the case for the USB cable. I also 3D printed out this little flexible grommet to help hold the cable in place. And yeah, sure, it's not gorgeous or anything, but it looks better than the front, so I'm cool with it. I popped that on this little 3D printed stand where I was able to also hook up the Ethernet adapter. And after that, I had this nice, tidy, compact setup that was only drawing one watt at idle. And I've only been running this for a few days, but I've yet to have any crashes or dropouts or any issues. And also, one watt at idle is kind of crazy. You could be running this 24/7 somewhere where power is like 50 cents per kilowatt hour and still only spend like $4 a year, which is pretty nuts. Oh, and I almost forgot, but I also ran stress just to make sure that the power adapter wouldn't be drawing too much power, but still the phone only drew around 6 watts. So, in the end, I am surprisingly happy with where I ended up. It's much better than I expected, honestly. Now, to be clear, well, I don't have to say that probably because you've watched this whole video, so you know, this is not just a simple plug-and-play process. If you're much smarter than I am when it comes to Android partitioning and flashing, and you're also familiar with configuring and using Linux operating systems, then this might be a smoother experience for you. But even then, getting this to do exactly what you want is probably going to take quite a bit of work. And even if you really do know what you're doing, there's a decent chance you might just run into some brick walls because, well, software support for ARM devices can be pretty tricky for even some popular platforms. And well, this definitely isn't a really popular platform. And that's just the software side of things. You're probably going to have to do some work to get power figured out and not end up with a spicy pillow. Also, even if you do get this working, you're still going to be a lot more limited uh in comparison to like x86 systems in terms of expandability and IO. Even if you have a USB hub or something, you're still limited to USB and probably USB 2.0.

That being said, if your device is supported and you want to try to get hacky and get it running on your device, this can actually be a really cool way to repurpose an old phone. I mean, heck, in the end, I ended up with a super low power dead silent system that can run Home Assistant and probably some other tasks. And even considering how much work it took to get there, the whole process was actually pretty fun. Well, minus the minus the NF tables part, that part really stunk. But hey, I learned a lot. And speaking of learning a lot, if you have any advice or tips or you've done this before, uh, post your experience down in the comments. It might be helpful for others and also me. I do want to give a massive shout out to everyone who contributes to Postmarket OS. It's a really cool project. It's some really cool software. And yeah, it's clunky. It has some issues, but it's honestly really cool what you could do with these old devices. So, shout out to anyone who's helped work on that. Also, massive shout out to you for making it this far in the video. I would assume you liked it. So, if so, maybe consider giving it a like, maybe consider subscribing for more, or maybe even consider becoming a RAID member for as little as a dollar a month. With that, you get early access to my videos without any ads, which I think is a pretty good deal. And if you want to jump up to a Raid five or Raid 10 membership, you can also get some access to some other cool things, like for example a recent behind the scenes video where I talked about some projects that I've been working on. I think that's about it for this one, though. So, as always, thank you guys so much for watching. Stay curious, and I really can't wait to see you in the next one. [Music]